Virtual Session: Remote Phone Access to the Local App

Design and implementation guide for letting a user open the locally running laptop app from a phone browser. It uses a **random Cloudflare quick-tunnel link** and **biometric login** (Face ID or fingerprint). The session stays active until it is deleted on the laptop or disconnected on the phone.

---

1. What it does

Requirement	How it is met
App runs locally on the laptop	The app never moves. A tunnel process on the laptop makes an **outbound** connection to Cloudflare. No port forwarding or public IP is needed.
"Create Virtual Session" gives a link	The laptop starts `cloudflared`, gets a random `https://xxxx.trycloudflare.com` URL, and shows it as a link and QR code.
Opens from any browser	The phone only needs Safari or Chrome. Nothing to install.
Biometric security is mandatory	WebAuthn passkeys with `userVerification: required`. The fingerprint or face never leaves the phone.
Doesn't expire; ends only on delete or disconnect	The session is a server-side record with no TTL. It dies only through **Delete** (laptop) or **Disconnect** (phone).

---

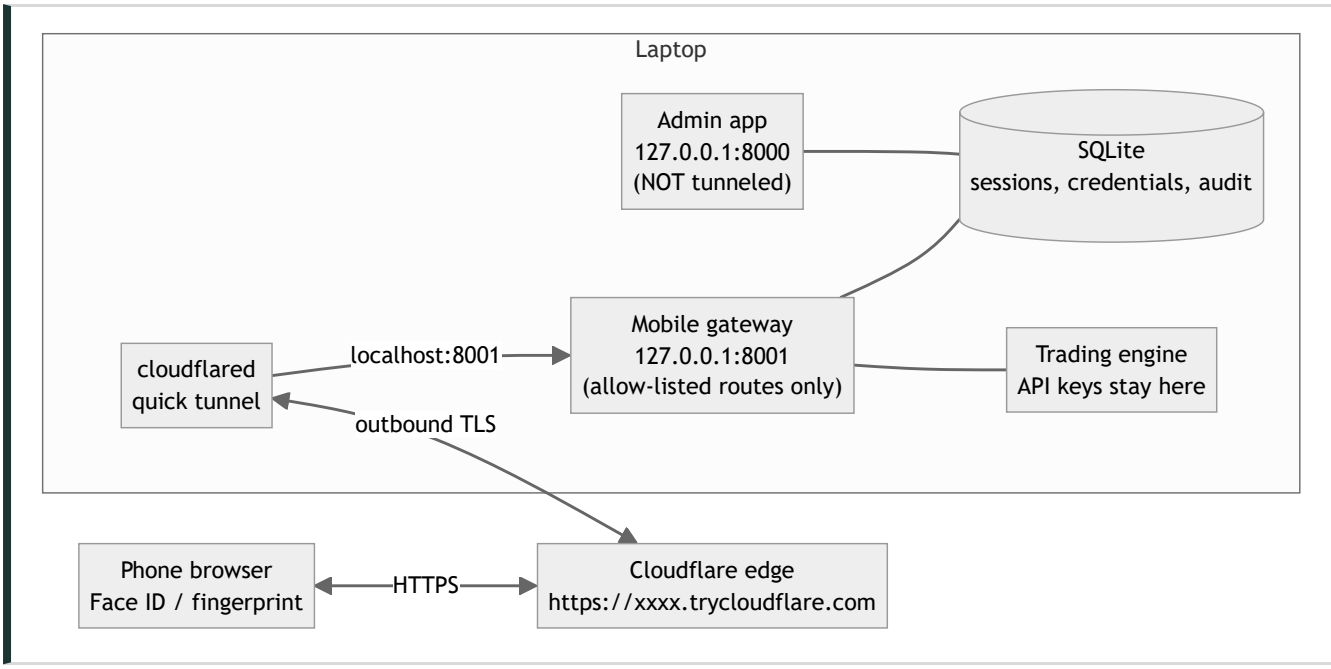
2. Important limitation of random URLs

> **Warning**
>
> Quick tunnels get a **new random hostname every time `cloudflared` starts**. Passkeys and browser storage are bound to the hostname, so this has consequences.
>
> - A session is persistent **only while the tunnel process keeps running**.
> - If the laptop restarts, sleeps long enough to drop the tunnel, or `cloudflared` crashes, the URL changes. The old link and passkey stop working.
> - The user then clicks **Reconnect** on the laptop, approves, and re-registers the biometric on the new link. This takes about 10 seconds.
> - Cloudflare also documents quick tunnels as test-grade: no uptime guarantee, a 200 concurrent-request limit, and no Server-Sent Events. Use WebSockets or polling for live updates. Verify these limits against the current Cloudflare docs.

The code reads the hostname from one config value (`RP_ID`). If this becomes a problem later, switch to a stable hostname with no redesign (see section 10).

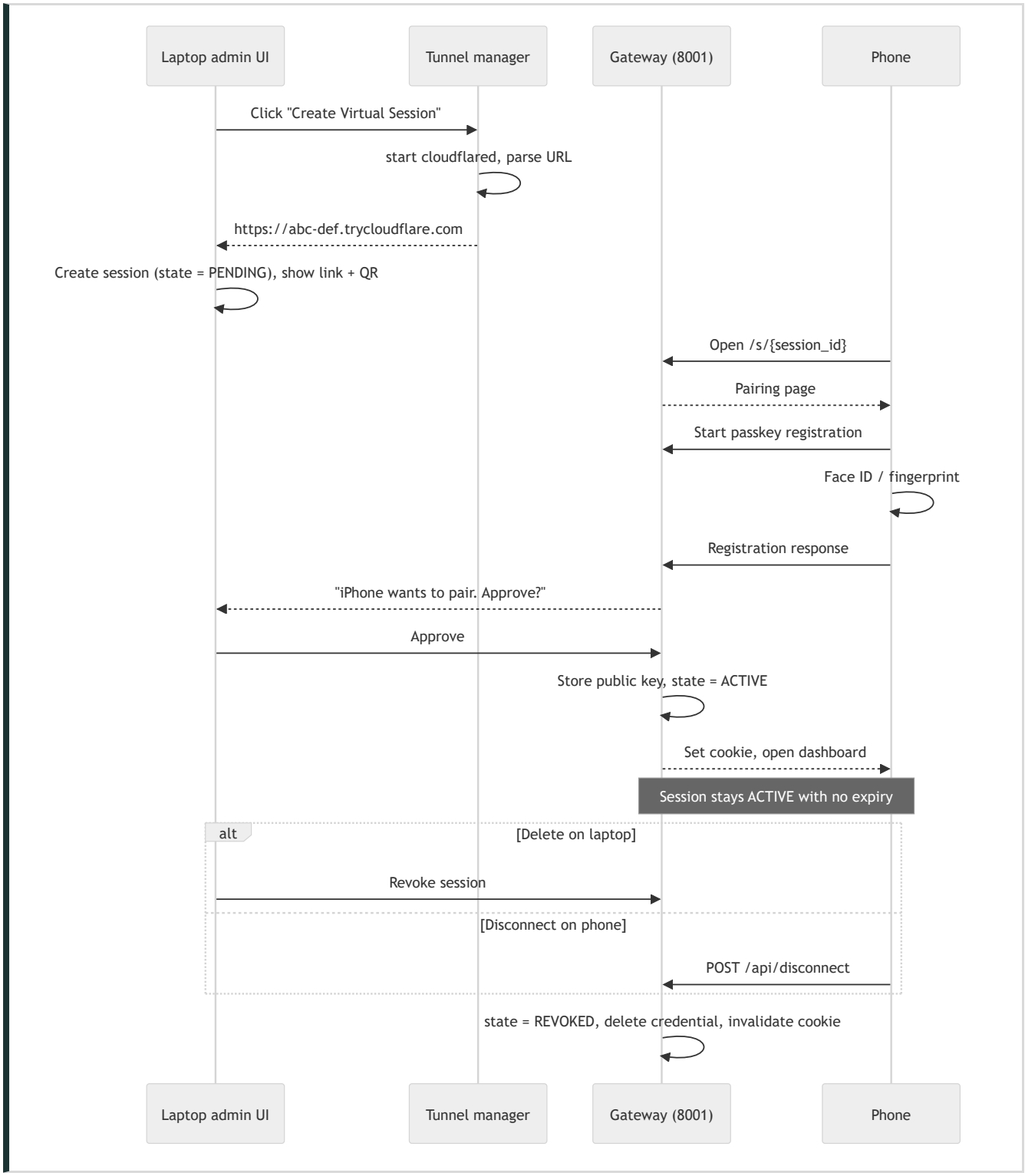
---

3. Architecture



Two separate servers is the most important design decision. Requests arriving through the tunnel appear to come from `127.0.0.1`, so you can't tell local from remote by IP. Only the **gateway** port (8001) is tunneled, and it exposes a minimal API. Admin functions (create, delete sessions, view API keys, change settings) live only on port 8000, which is never exposed.

4. Lifecycle


Laptop admin UI
Tunnel manager
Gateway (8001)
Phone
Click "Create Virtual Session"
start cloudflared, parse URL
https://abc-def.trycloudflare.com
Create session (state = PENDING), show link + QR
Open /s/{session_id}
Pairing page
Start passkey registration
Face ID / fingerprint
Registration response
"iPhone wants to pair. Approve?"
Approve
Store public key, state = ACTIVE
Set cookie, open dashboard
Session stays ACTIVE with no expiry
alt
[Delete on laptop]
Revoke session
[Disconnect on phone]
POST /api/disconnect
state = REVOKED, delete credential, invalidate cookie
Laptop admin UI
Tunnel manager
Gateway (8001)
Phone

Session states

State	Meaning
`PENDING`	Link created. Not yet claimed or approved. Only the pairing page is reachable.
`ACTIVE`	Paired and approved. The phone can use the app.
`RELINK_REQUIRED`	Tunnel URL changed. The old passkey is unusable. The laptop shows a **Reconnect** button.
`REVOKED`	Deleted or disconnected. Terminal. Everything is wiped.

---

5. Security model

1. **The link alone is not a credential.** Opening it only leads to a biometric prompt.
2. **Laptop approval at pairing.** A first device can't silently claim a link. The laptop shows the device and a 6-digit confirmation code that the phone must display. This closes the gap left by having no link expiry.
3. **Passkey with required user verification.** The server rejects any assertion where the user-verified flag is not set.
   - WebAuthn guarantees "user verified", which is biometrics **or** the phone's PIN. It cannot force biometrics only.
4. **Cookie bound to the passkey.** Session cookies are random 256-bit values, stored hashed on the server, `HttpOnly; Secure; SameSite=Strict`. Revocation is a database flag, so it takes effect instantly. Because the session has no expiry, a stolen cookie is only useful if it is also tied to the registered credential.
5. **Step-up biometric for sensitive actions.** Placing or changing trades, or enabling live mode, requires a fresh passkey assertion. The challenge includes a hash of the action, so a prompt for one action can't authorize another. A step-up token is valid for about 60 seconds and single-use.
6. **Optional app-lock (a re-lock, not a logout).** After the phone tab has been in the background, ask for biometrics again. This is a setting.
7. **API keys never reach the phone.** Exchange keys stay on the laptop, and withdrawal permission should be disabled on the keys themselves.
8. **Mobile is read-only by default.** Trade actions are opt-in per session.
9. **Rate limiting and lockout.** Limit login attempts per session and per IP (`Cf-Connecting-Ip` header). Lock a session after repeated failures.
10. **Audit log.** Record session creation, approval, every login, step-up, trade, and revocation, with time, IP and country (`Cf-Ipcountry`). Show it on the laptop.
11. **Kill switch.** One laptop button stops mobile trading and revokes all sessions. Closing the app stops the tunnel, which ends all access.

---

6. Data model (SQLite)

```
CREATE TABLE sessions (
  id            TEXT PRIMARY KEY,        -- random 128-bit, in the link path
  name          TEXT NOT NULL,           -- "My iPhone"
  state         TEXT NOT NULL,           -- PENDING|ACTIVE|RELINK_REQUIRED|REVOKED
  tunnel_host   TEXT NOT NULL,           -- hostname when created; equals WebAuthn rpID
  pair_code     TEXT,                    -- 6-digit code for laptop approval
  can_trade     INTEGER DEFAULT 0,
  created_at    INTEGER NOT NULL,
  approved_at   INTEGER,
  last_seen_at  INTEGER,
  last_ip       TEXT,
  last_country  TEXT,
  user_agent    TEXT
);

CREATE TABLE credentials (
  session_id    TEXT REFERENCES sessions(id) ON DELETE CASCADE,
  credential_id BLOB PRIMARY KEY,
  public_key    BLOB NOT NULL,
  sign_count    INTEGER NOT NULL,
  transports    TEXT
);

CREATE TABLE cookies (
  token_hash    BLOB PRIMARY KEY,        -- SHA-256 of the cookie value
  session_id    TEXT REFERENCES sessions(id) ON DELETE CASCADE,
  created_at    INTEGER NOT NULL
);

CREATE TABLE challenges (                -- short-lived, single-use
  id            TEXT PRIMARY KEY,
  session_id    TEXT,
  purpose       TEXT,                    -- register|login|stepup
  challenge     BLOB NOT NULL,
  action_hash   BLOB,                    -- binds step-up to one action
  expires_at    INTEGER NOT NULL         -- ~2 minutes
);

CREATE TABLE audit_log (
  ts INTEGER, session_id TEXT, event TEXT, ip TEXT, country TEXT, detail TEXT
);
```

The 2-minute expiry applies only to the one-time WebAuthn **challenges**, not to the session.

7. API

Admin app (127.0.0.1:8000, never tunneled)

Method	Path	Purpose
POST	`/admin/sessions`	Create a session. Ensures the tunnel is up and returns link and QR.
GET	`/admin/sessions`	List sessions: name, state, device, last seen, IP and country.
POST	`/admin/sessions/{id}/approve`	Approve a pending pairing (after the code matches).
POST	`/admin/sessions/{id}/reconnect`	Issue a new link for `RELINK_REQUIRED`.
DELETE	`/admin/sessions/{id}`	Revoke and wipe.
POST	`/admin/killswitch`	Revoke everything and stop the tunnel.

Mobile gateway (127.0.0.1:8001, tunneled, allow-listed)

Method	Path	Auth	Purpose
GET	`/s/{id}`	none	Pairing or login page, depending on state
POST	`/webauthn/register/options` and `/verify`	link only, state = PENDING	Register passkey
POST	`/webauthn/login/options` and `/verify`	credential	Biometric login, issues cookie
POST	`/webauthn/stepup/options` and `/verify`	cookie	Fresh biometric for a sensitive action
GET	`/api/dashboard`, `/api/positions`, `/api/signals`	cookie	Read-only data
POST	`/api/trade/*`	cookie + step-up token + `can_trade`	Trade actions
POST	`/api/disconnect`	cookie	Phone-side revoke
WS	`/ws`	cookie	Live updates (not SSE)

Anything not listed returns 404. Never proxy the whole local app.

---

8. Implementation guide

Suggested stack, assuming a Python app. Adapt it if the app is Node or Electron.

- **Web framework:** FastAPI (two `uvicorn` instances, ports 8000 and 8001)
- **Passkeys:** `webauthn` (py_webauthn) library
- **Database:** SQLite
- **Tunnel:** the `cloudflared` binary, run as a subprocess
- **QR code:** `qrcode` (Python) rendered in the admin UI

- **Phone UI:** a plain responsive page or PWA, with `@simplewebauthn/browser` or the native `navigator.credentials` API

8.1 Tunnel manager

```
# tunnel.py
import re, subprocess, threading, time

URL_RE = re.compile(r"https://[a-z0-9-]+\.trycloudflare\.com")

class TunnelManager:
    def __init__(self, cloudflared_path="cloudflared", port=8001):
        self.path, self.port = cloudflared_path, port
        self.proc, self.url = None, None
        self.on_url_change = None          # callback: (old_url, new_url)

    def start(self, timeout=30):
        if self.proc and self.proc.poll() is None:
            return self.url
        self.proc = subprocess.Popen(
            [self.path, "tunnel", "--no-autoupdate", "--url", f"http://127.0.0.1:{self.port}"],
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        old, self.url = self.url, None
        t0 = time.time()
        for line in self.proc.stdout:          # cloudflared logs the URL here
            m = URL_RE.search(line)
            if m:
                self.url = m.group(0)
                break
            if time.time() - t0 > timeout:
                raise TimeoutError("cloudflared did not return a URL")
        threading.Thread(target=self._drain_and_watch, daemon=True).start()
        if old and old != self.url and self.on_url_change:
            self.on_url_change(old, self.url)   # mark sessions RELINK_REQUIRED
        return self.url

    def _drain_and_watch(self):
        for _ in self.proc.stdout: pass         # keep the pipe from filling up
        # process exited, so the URL is gone; the watchdog restarts it

    def stop(self):
        if self.proc: self.proc.terminate(); self.proc = None; self.url = None
```

Start the tunnel when the first session is created or when the app launches. Add a watchdog that restarts it if it exits. On a URL change, set every non-revoked session to `RELINK_REQUIRED`.

8.2 WebAuthn configuration (hostname-driven)

```
from urllib.parse import urlparse
def rp_config(session):                        # hostname comes from the session, not hardcoded
    host = session["tunnel_host"]
    return {"rp_id": host, "origin": f"https://{host}", "rp_name": "PegGuard"}
```

- **Registration options:** `authenticator_attachment="platform"`, `user_verification="required"`, `resident_key="preferred"`.
- **Verify** with `expected_rp_id` and `expected_origin` from `rp_config`.
- **Login:** `allow_credentials` is the session's credential only. After verification, check the user-verified flag and that `sign_count` increased, then issue a cookie.

- **Step-up:** same as login, but the challenge is `HMAC(server_secret, action_hash || nonce)` and the result is a 60-second single-use token.

8.3 Gateway hardening

- Respond 404 to every route not in the allow-list above.
- Middleware on every request:
- reject if the session is `REVOKED` or `RELINK_REQUIRED`;
- verify the `Host` header equals the session's `tunnel_host`;
- apply rate limits;
- update `last_seen`, `last_ip`, `last_country`.
- Headers: `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store`.
- Never log tokens, cookies or API keys.

8.4 Laptop UI ("Virtual Sessions" screen)

- **Create Virtual Session** button, a name field, and an optional "allow trading" toggle.
- Link, copy button and QR code. Pending sessions show a **device wants to pair** card with the 6-digit code and **Approve / Reject**.
- A list of sessions with state badge, device, last seen, location and a **Delete** button. `RELINK_REQUIRED` sessions get a **Reconnect** button.
- An audit log tab and a red **Kill switch** button.
- A banner: "Keep this laptop awake." Optionally disable sleep while sessions are active (Windows: `SetThreadExecutionState`).

8.5 Phone UI

- **Pairing page:** shows the 6-digit code, then the biometric prompt.
- **Dashboard:** read-only by default, with a visible **Disconnect** button and a connection indicator.
- **Offline state:** if the WebSocket drops, show "Laptop offline. Reconnecting…".
- **Disconnect:** calls `/api/disconnect`, clears the cookie, and shows "Disconnected. Ask for a new link."
- Optional PWA manifest so it can be added to the home screen.

---

9. Edge cases

Case	Behavior
Laptop sleeps or shuts down	Link is dead. The phone shows an error. On wake, the tunnel may get a new URL, so sessions become `RELINK_REQUIRED`.
`cloudflared` crashes	The watchdog restarts it. The URL changes, so sessions are marked `RELINK_REQUIRED` and the laptop shows **Reconnect**.
Phone lost or stolen	Delete the session on the laptop. Biometrics also protect the phone.
Delete on laptop while the phone is open	The next request returns 401, and the phone shows "Session ended".
Link leaks before pairing	Pairing still needs laptop approval and the matching code. Unapproved pending sessions can be rejected or deleted.
Biometrics not enrolled on the phone	Registration fails with a clear message. Don't downgrade to a password.
Two devices	Create one session per device. Each is deleted independently.
Phone browser clears site data	The cookie and passkey reference are lost. Treat it as a new device and re-pair.

10. Optional upgrade: stable URL

If re-pairing after each restart becomes annoying, change only the tunnel manager and `RP_ID` source:

- **Tailscale Funnel:** gives a stable, free `https://<machine>.<tailnet>.ts.net` URL that works in any phone browser. Confirm current Funnel availability and limits on the free plan.
- **Cloudflare named tunnel:** a stable hostname on a domain you own (about $10 a year). Also supports Cloudflare Access as an extra login layer.

With a stable hostname, sessions survive restarts and `RELINK_REQUIRED` almost never happens.

11. Build plan

Phase	Deliverable	Rough effort
1	Split admin (8000) and gateway (8001). Add the SQLite schema and the tunnel manager.	0.5–1 day
2	Create-session flow, link and QR, pairing page, laptop approval with the 6-digit code.	1 day
3	WebAuthn registration and login, cookie issuance, revocation (laptop Delete and phone Disconnect).	1–2 days
4	Read-only dashboard, WebSocket updates, offline and relink handling.	1 day
5	Step-up biometrics for trading, audit log, rate limits, kill switch.	1–2 days
6	Testing: real iPhone and Android, URL-change recovery, revocation races, security review.	1–2 days

Test checklist

- Pair on iOS Safari and Android Chrome. Biometric prompt appears, registration succeeds.
- Copy the link to another device: it can't pair without laptop approval and the code.
- Delete on laptop: the phone loses access on its next request.
- Disconnect on phone: the laptop list updates and the credential is wiped.
- Kill and restart `cloudflared`: sessions become `RELINK_REQUIRED` and Reconnect works.
- An admin route requested through the tunnel returns 404.
- A trade without a fresh step-up token is rejected. A replayed step-up token is rejected.
- Exchange keys never appear in any gateway response.